$ cd aula-js && code .

Arrays, Objetos e Funções

JavaScript na prática, direto no VS Code

aprovados.js

```
const turma = [
  { nome: "Ana", nota: 8 },
  { nome: "Bruno", nota: 5 },
  { nome: "Carla", nota: 9 }
];
function aprovados(lista) {
  const nomes = [];
  for (const aluno of lista) {
    if (aluno.nota ⩾ 6) {
      nomes.push(aluno.nome);
    }
  }
  return nomes;
}
aprovados(turma);
// ["Ana", "Carla"]
```

Profa. Ma. Tamara Simões · UC4 Desenvolvimento Front-end · Faculdade Senac Contagem

O que vamos fazer hoje

01

Preparar o VS Code

Pasta, arquivo e terminal prontos para rodar JavaScript.

02

Arrays

A lista do Python: criar, alterar e percorrer.

03

Objetos

O dicionário do Python, agora com ponto.

04

Funções

De def para function, e o atalho da arrow.

{ } No final: um desafio que junta os três e a ponte direta para o jogo do TDE 4.

Preparando o VS Code

1. Instale a extensão Code Runner no VS Code.
2. Crie a pasta aula-js-3 e abra no VS Code (Arquivo › Abrir Pasta).
3. Crie o arquivo script.js dentro dela.

Lembre-se: o arquivo deve ser salvo antes de rodar.

Sem Node instalado?

Crie um index.html com <script src="script.js"></script>, abra com o Live Server e veja a saída no Console do navegador (F12).

script.js

```
console.log("Olá, turma!");
```

Se o Code Runner não estiver instalado?

```
$ node -v
v20 ou mais novo
$ node script.js
Olá, turma!
```

// 01

Arrays

A lista do Python, com outro sobrenome.

Lista no Python, array no JS

Python

```
frutas = ["maçã", "banana", "uva"]
print(frutas[0]) # maçã
print(len(frutas)) # 3
```

JavaScript

```
const frutas = ["maçã", "banana", "uva"];
console.log(frutas[0]); // maçã
console.log(frutas.length); // 3
```

Índice começa em 0

Igual ao Python: o primeiro é o `[0]`.

len() vira .length

É propriedade: vem sem parênteses.

[-1] não funciona

Para o último, use `frutas.at(-1)`.

const não congela

Os itens mudam; o array inteiro, não.

Os métodos que você vai usar sempre

O que fazer	Python	JavaScript
Adicionar no fim	`lista.append(x)`	`array.push(x)`
Remover do fim	`lista.pop()`	`array.pop()`
Adicionar no início	`lista.insert(0, x)`	`array.unshift(x)`
Remover do início	`lista.pop(0)`	`array.shift()`
Está na lista?	`x in lista`	`array.includes(x)`
Posição do item	`lista.index(x)`	`array.indexOf(x)`
Fatiar um pedaço	`lista[1:3]`	`array.slice(1, 3)`

não achou? indexOf dá -1

Percorrendo um array

Python

```python
notas = [8, 6.5, 9]

for nota in notas:
    print(nota)
```

o for...of é o mais parecido!

JavaScript

```javascript
const notas = [8, 6.5, 9];
// 1. for clássico, com índice
for (let i = 0; i < notas.length; i++) {
  console.log(notas[i]);
}

// 2. for...of, direto nos valores
for (const nota of notas) {
  console.log(nota);
}
```

Mão na massa: arrays

no arquivo `tarefas.js`

1. Crie `const tarefas` com 3 tarefas da sua semana.
2. Adicione mais uma no fim com `push`.
3. Remova a primeira com `shift` e mostre qual saiu.
4. Mostre quantas tarefas restam com `length`.
5. Percorra com `for...of` e mostre cada tarefa.

8
minutos

// 02

Objetos

O dicionário do Python, agora com ponto.

Dicionário no Python, objeto no JS

Python

```python
aluno = {
    "nome": "Ana",
    "idade": 19,
    "curso": "ADS"
}
print(aluno["nome"]) # Ana
print(aluno["idade"]) # 19
```

JavaScript

```javascript
const aluno = {
  nome: "Ana",
  idade: 19,
  curso: "ADS"
};
console.log(aluno.nome); // Ana
console.log(aluno["idade"]); // 19
```

Chave sem aspas

Nome simples dispensa as aspas.

Ponto ou colchete

aluno.nome e aluno["nome"] dão no mesmo.

Chave numa variável?

Aí só o colchete: aluno[campo].

Mexendo no objeto

O que fazer	Python	JavaScript
Adicionar ou alterar	`aluno["idade"] = 20`	`aluno.idade = 20;`
Remover	`del aluno["curso"]`	`delete aluno.curso;`
A chave existe?	`"nome" in aluno`	`"nome" in aluno`
Percorrer as chaves	`for chave in aluno:`	`for (const chave in aluno)`
Lista das chaves	`list(aluno.keys())`	`Object.keys(aluno)`
Lista dos valores	`list(aluno.values())`	`Object.values(aluno)`

{ } Regra de bolso: for...in para objetos, for...of para arrays.

Array de objetos: a turma inteira

```JavaScript
const turma = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 6 },
  { nome: "Carla", nota: 9.2 }
];

for (const aluno of turma) {
  console.log(`${aluno.nome}: ${aluno.nota}`);
}

console.log(turma[1].nome); // Bruno
```

Cada item é um objeto

Índice acha o aluno, ponto acha o dado.

${ } é o f-string do JS

Só funciona entre crases, nunca entre aspas.

É o formato do JSON

Dados de APIs chegam assim no front-end.

Mão na massa: objetos

no arquivo `perfil.js`

1. Crie `const eu` com nome, curso e linguagens (um array).
2. Adicione a propriedade `cidade`.
3. Mostre a segunda linguagem: `eu.linguagens[1]`.
4. Percorra com `for...in` mostrando chave e valor.
5. Bônus: crie `colegas`, um array com 3 objetos, e mostre só os nomes.

8
minutos

// 03

Funções

O def vira function, e o resto você já sabe.

def no Python, *function* no JS

Python

```python
def media(a, b):
    return (a + b) / 2

print(media(7, 9)) # 8.0
```

JavaScript

```javascript
function media(a, b) {
  return (a + b) / 2;
}
console.log(media(7, 9)); // 8
```

{ } no lugar do recuo

As chaves marcam o bloco; o recuo é só para ler.

Sem return, undefined

É o None do Python.

Valor padrão igualzinho

function saudar(nome = "turma")

Três jeitos de escrever uma função

Declaração

```
function dobro(n) {
  return n * 2;
}
```

A forma clássica. Pode ser chamada antes de aparecer no código.

Expressão

```
const dobro = function (n) {
  return n * 2;
};
```

A função vira um valor guardado numa variável.

Arrow function

```
const dobro = (n) ⇒ n * 2;

// Python: lambda n: n * 2
```

A mais curta e a mais vista no front-end. Prima do lambda.

`dobro(5)` As três se chamam do mesmo jeito e devolvem 10.

Juntando tudo

boletim.js

```
const turma = [
  { nome: "Ana", notas: [8, 9, 7.5] },
  { nome: "Bruno", notas: [5, 6.5, 4] }
];
function media(notas) {
  let soma = 0;
  for (let i = 0; i < notas.length; i++) {
    soma += notas[i];
  }
  return soma / notas.length;
}
for (const aluno of turma) {
  const m = media(aluno.notas);
  const situacao = m ⩾ 6 ? "aprovado" : "recuperação";
  console.log(`${aluno.nome}: ${m.toFixed(1)} (${situacao})`);
}
```

Terminal

```
$ node boletim.js
Ana: 8.2 (aprovado)
Bruno: 5.2 (recuperação)
```

? : é o if/else curto

No Python: x if cond else y

toFixed(1)

Uma casa decimal, como round(m, 1).

Desafio: a lojinha

no arquivo `loja.js`

1. Crie `produtos`, um array com 4 objetos: nome, preco e estoque.
2. Escreva `emEstoque(lista)`, que devolve só os produtos com estoque maior que 0.
3. Escreva `total(lista)`, que soma os preços.
4. Mostre cada produto em estoque assim: Caderno: R$ 12.50
5. Bônus: com um `for`, gere a lista de preços com 10% de desconto.

Desafio: ponto de partida

loja.js

```
const produtos = [
  { nome: "Caderno", preco: 12.5, estoque: 3 },
  // + 3 produtos, um com estoque 0
];

function emEstoque(lista) {
  // sua vez
}

function total(lista) {
  // sua vez
}
```

emEstoque

Array vazio, um for, um if e push.

total

Uma variável em 0 e um for somando cada preço.

Formatar o preço

toFixed(2) mostra 12.50.

Onde isso entra no jogo do TDE 4

Array

Inimigos, obstáculos e moedas na tela.

```
const inimigos = [];
inimigos.push(novoInimigo);
```

Objeto

O jogador, com posição e estado.

```
const jogador = {
  x: 100, y: 300,
  vidas: 3, pontos: 0
};
```

Função

Cada ação do jogo vira uma função.

```
function mover(p, dx) {
  p.x += dx;
}
```

A cada quadro, percorremos o array e chamamos as funções:

```
for (const inimigo of inimigos) mover(inimigo, -2);
```

Colinha: Python → JavaScript

Python	JavaScript	Python	JavaScript
`print("oi")`	`console.log("oi")`	`d = {"a": 1}`	`const obj = { a: 1 };`
`lista = [1, 2]`	`const lista = [1, 2];`	`d["a"]`	`obj.a`
`len(lista)`	`lista.length`	`def f(x):`	`function f(x) { }`
`lista.append(3)`	`lista.push(3)`	`lambda x: x * 2`	`(x) ⇒ x * 2`
`for x in lista:`	`for (const x of lista)`	`f"Oi, {nome}"`	`` `Oi, ${nome}` ``
`x in lista`	`lista.includes(x)`	`None`	`undefined`

tira print dessa!